



AdvencedIfStatements

🕒 Created time	July 18, 2026 6:58 AM
☰ Category	Conditionals
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 19, 2026 4:10 AM

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Professor Corey");
}
else if (firstName.ToLower() == "tim" ||
        lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great part in your name.");
}
else
{
    Console.WriteLine("Hello Student.");
}

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}
else if (firstName.ToLower() == "tim")
{
```

```

        Console.WriteLine("Hello, you have a cool first name.");
    }
    else if (firstName.ToLower() == "corey")
    {
        Console.WriteLine("You have a great last name.");
    }
    else
    {
        Console.WriteLine("Sorry you don't have a cooler name!");
    }
    //~~~~~

```

```

if (firstName.ToLower() == "tim")
{
    Console.WriteLine("You have a cool first name.");
}

if (lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}
else
{
    Console.WriteLine("Sorry your name isn't cooler.");
}
//~~~~~

```

```

int age = 73;

// ==, >, >=, <, <=, != (! =)->means not equal
if (age != 43)
{
    Console.WriteLine("Sorry ,you aren't a great age.");
}

```

```

if ((age >= 40 && age < 50) ||
    (age >= 70 && age < 80))

```

```
{
    Console.WriteLine("You are in your 40's or 70's");
}
```

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();
Console.Write("What is your last name; ");
string lastName = Console.ReadLine();
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello Mr. Corey.");
}
```

ה- && (And)

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" && lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}
```

השם הפרטי ושם המשפחה חייבים להחזיר ערך **true**, אם זה המצב, תבוצע הדפסה של

```
"Hello Mr. Corey."
```

שימו לב, יכולים להיות לנו הרבה תנאים כאלה, אבל השורה תהיה ארוכה מאוד.

אנחנו יכולים לשים שבירת שורה ו-**#C** תבין זאת מכיוון שהיא קוראת לפי הפתיחה והסגירה של הסוגריים **()**.
 וזה עוזר לנו מבחינה לוגית, כי קל מאוד לקרוא שורה אחר שורה בצורה הזו:

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
```

```
Console.WriteLine("Hello Mr. Corey.");
}
```

אל תעשו את שבירת השורה הזו, זו שבירת שורה גרועה:

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine(
        "Hello Mr. Corey."
    );
}
```

~~~~~

## ה-else if

הצהרת ה-else מתרחשת אם הדבר הראשון הזה הוא false, כלומר לא "tim" ולא "corey":

```
if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}
```

אז אנחנו יכולים לומר else if:

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();
```

```

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}
else if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello, you have a cool first name.");
}

```

אם נקליד "corey" ו-"tim", הקטע הזה לא ירוץ:

```

else if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello, you have a cool first name.");
}

```

כי "tim" ו-"corey" שניהם **true**.  
 הצהרת ה-**else if** הזו משמעותה אם מה שמעל הוא **false**.  
 אז זה יעצור אחרי זה:

```

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}

```

~~~~~

הוספת תנאי **else if** נוסף

אנחנו יכולים להוסיף עוד משפט **else if** למבנה התנאים שלנו:

```

Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")

```

```

{
    Console.WriteLine("Hello Mr. Corey.");
}
else if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello, you have a cool first name.");
}
else if (firstName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}

```

מה שירוצ בפועל

קטע הקוד הבא ירוץ:

```

else if (firstName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}

```

מתי הקטע הזה ירוץ?

אך ורק אם התנאי הקודם לו יצא **false**:

```

else if (firstName.ToLower() == "tim")

```

למה זה עובד בצורה הזו?

מכיוון שאם התנאי הזה יצא **true**:

```

else if (firstName.ToLower() == "tim")

```

התנהגות התוכנית כשהתנאי מתקיים

התוכנית תריץ את קטע הקוד הזה, ותסתיים מיד.

היא לא תריץ שום דבר אחר שנמצא בהמשך השרשרת:

```

{
    Console.WriteLine("Hello, you have a cool first name.");
}

```

כללים חשובים לתנאים **Chaining**

שימו לב, במבנה תנאים משורשר יכולים להיות לנו הרבה מאוד משפטי **else if**.

יכול להיות לנו אך ורק בלוק **else** אחד בסוף השרשרת.

יכול להיות לנו אך ורק משפט **if** אחד בתחילת השרשרת, כאשר כולם משורשרים ומחוברים יחד למבנה לוגי אחד.

~~~~~

## דוגמה לקוד לא תנאים :Chained Together

```
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("You have a cool first name.");
}
if (lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}
else
{
    Console.WriteLine("Sorry your name isn't cooler.");
}
```

אם נריץ את הקוד הזה ונזין את השם **"Corey"** **"Tim"**, המערכת תדפיס:

"You have a great last name." וגם "You have a cool first name."

הסיבה ששתי ההודעות מודפסות היא שהתנאים אינם **chained** זה לזה.

כדי שהם יהיו **chained** ביניהם וליצור תלות, אנחנו חייבים להשתמש ב-**else if**:

```
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("You have a cool first name.");
}
else if (lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}
else
{
    Console.WriteLine("Sorry your name isn't cooler.");
}
```

כאשר נריץ את הקוד הזה, המערכת תדפיס רק:

```
"You have a cool first name."
```

אבל מה לגבי שם המשפחה?

מכיוון שהבדיקה של שם המשפחה נמצאת בתוך בלוק של **else if**, המשמעות היא שהיא תרוץ אך ורק אם התנאי שמעליה (**בדיקת השם הפרטי**) החזיר **false**. במקרה הזה, התנאי של השם הפרטי היה **true**, ולכן המערכת דילגה לחלוטין על הבדיקה של שם המשפחה.

**שימו לב:**

לפעמים אנחנו דווקא נצטרך שני משפטי **if** נפרדים.

הסיבה לכך היא שנרצה להריץ שני קטעי קוד שונים במקרה שנתונים שונים תואמים באופן בלתי תלוי, והם נפרדים בדיוק בצורה הזו:

```
if (firstName.ToLower() == "tim")
{
    Console.WriteLine("You have a cool first name.");
}

if (lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}
else
{
    Console.WriteLine("Sorry your name isn't cooler.");
}
```

נשתמש במבנה הזה כאשר אנחנו צריכים לבדוק גם את השם הפרטי וגם את שם המשפחה, ללא תלות ביניהם.

~~~~~

else - ה

ה-**else** הזה משמש ל**ניקוי**, סגירת פינות וטיפול במצבים שגוים. אם אף אחד מהתנאים האחרים לא רץ, ה-**else** הזה ירוץ:

```
Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();
```



```

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Mr. Corey.");
}
else if (firstName.ToLower() == "tim")
{
    Console.WriteLine("Hello, you have a cool first name.");
}
else if (firstName.ToLower() == "corey")
{
    Console.WriteLine("You have a great last name.");
}
else
{
    Console.WriteLine("Sorry you don't have a cooler name!");
}

```

המשמעות המקצועית בקוד

מושג ה-**Clean up** נקרא בשפת הפיתוח המקצועית **Fallback** או **Catch-all**, רשת ביטחון.

התפקיד של ה-**else** האחרון הוא לוודא שהקוד שלנו לא משאיר קצוות פתוחים.

גם אם המשתמש הזין קלט שלא תואם לאף אחד מהתנאים הספציפיים שבדקנו קודם לכן (**if-וב-else if**), המערכת תדע בדיוק לאן לנתב את הריצה ולא תישאר ללא תגובה.

ככה בונים קוד יציב שמונע התנהגויות לא צפויות ובאגים במערכת.

~~~~~

### דוגמה: טעות נפוצה ללא שימוש ב

.ToLower

```

if (firstName = "tim")
{
    Console.WriteLine("You have a cool first name.");
}

```

סביבת הפיתוח **Visual Studio** מציגה לנו שגיאה שאומרת:

"Cannot implicitly convert type 'string' to 'bool'"

לא ניתן להמיר באופן מרומז מטיפוס **string** לטיפוס **bool**.

את השגיאה הזו קשה להבין במבט ראשון.

השגיאה נובעת מכך שכתבנו סימן שווה אחד (=) בלבד.

סימן שווה אחד מהווה בעיה מכיוון שמדובר באופרטור Assignment, מה שאומר שהקוד בעצם מכניס את הערך "tim" לתוך המשתנה

```
firstName
```

לאחר מכן, המערכת מנסה להעריך את המשתנה **firstName** כדי לבדוק אם הוא **true** או **false**, כמו שנדרש בתנאי **if**, אבל מכיוון שהוא מטיפוס **string**, הוא מודיע שלא ניתן להמיר **string** לערך **bool**.

**שימו לב:** קוד שעובד טכנית, אבל נחשב לקוד גרוע

```
bool isComplete = false;
if (isComplete = true)
{
    Console.WriteLine("You have a cool first name.");
}
```

הסיבה שזהו קוד גרוע היא שלא מתבצעת כאן השוואה.

במקום להשוות, אנחנו מכניסים את הערך **true** לתוך המשתנה **isComplete**, ורק לאחר מכן מתבצעת ההערכה של המשתנה **isComplete** עצמו.

אם נריץ את הקוד הזה, יודפס בפלט:

```
"You have a cool first name."
```

זה קורה בגלל ש-**isComplete** שווה כעת ל-**true**, מכיוון שביצענו השמה של הערך הזה קודם, ורק אז התנאי הוערך כ-**true**.

**כלל ברזל חשוב להבדלה**

סימן = אחד הוא אופרטור **Assignment** – תפקידו להכניס ערך לתוך משתנה.

שני סימני == הם אופרטור **Comparison** – תפקידם לבדוק האם שני ערכים שווים זה לזה.

## הקוד הנכון והתקין

```
bool isComplete = false;
if (isComplete == true)
{
    Console.WriteLine("You have a cool first name.");
}
```

~~~~~

אפשרויות נוספות:

האופרטור **greater than <**

אם הגיל הוא 100:

```
int age = 100;
if (age > 100)
{
    Console.WriteLine("You are up there in years.");
}
```

לא תתבצע הדפסת **Console.WriteLine**.

אם הגיל הוא 101:

```
int age = 101;
if (age > 100)
{
    Console.WriteLine("You are up there in years.");
}
```

אנחנו מקבלים הדפסה למסך:

```
"You are up there in years."
```

זה קורה מכיוון שהשתמשנו באופרטור **greater then <**.

האופרטור **greater than or equals to =<**

אם הגיל הוא 100:

```
int age = 100;
if (age >= 100)
{
    Console.WriteLine("You are up there in years.");
}
```

אנחנו מקבלים הדפסה למסך:

```
"You are up there in years."
```

זה קורה מכיוון שהשתמשנו באופרטור **greater than or equals to <=**

האופרטור **less than or equals to >=**

אם הגיל הוא 100:

```
int age = 100;
if (age <= 100)
{
    Console.WriteLine("You still have a ways to go.");
}
```

אנחנו מקבלים הדפסה למסך:

```
"You still have a ways to go."
```

אם הגיל הוא 101:

```
int age = 101;
if (age <= 100)
{
    Console.WriteLine("You are up there in years.");
}
```

לא תתבצע הדפסת **Console.WriteLine**

~~~~~

## בדיקת Ranges

כאשר הגיל הוא 101

```
int age = 101;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אין פלט למסך **Console.WriteLine**.

הסיבה היא שאנחנו מחוץ ל **range**, מכיוון ש-101 גדול מ-50.

### כאשר הגיל הוא 43

```
int age = 43;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אנחנו מקבלים פלט למסך:

```
"You are in your 40's".
```

התנאי מתקיים מכיוון ש-43 נמצא בתוך הטווח המוגדר (גם גדול או שווה ל-40 וגם קטן מ-50).

### כאשר הגיל הוא 39

```
int age = 39;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אין פלט למסך **Console.WriteLine**

אנחנו מחוץ לטווח, מכיוון ש-39 קטן מ-40.

### כאשר הגיל הוא 49

```
int age = 49;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אנחנו מקבלים פלט למסך:

```
"You are in your 40's".
```

התנאי מתקיים מכיוון ש-49 עדיין קטן מ-50 ונמצא בתוך הגבול העליון של הטווח.

### כאשר הגיל הוא 40 (מקרה קצה תחתון)

```
int age = 40;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אנחנו מקבלים פלט למסך:

```
"You are in your 40's".
```

התנאי מתקיים בזכות האופרטור **greater than or equals to** (**>=**) שכולל את המספר **40** עצמו בתוך הטווח.

כאשר הגיל הוא **50** (מקרה קצה עליון)

```
int age = 50;
if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}
```

אין פלט למסך, אנחנו מחוץ לטווח.

התנאי לא מתקיים מכיוון שהשתמשנו באופרטור **less than** (**>**) ולא **less than or equals to** (**>=**), ולכן המספר **50** עצמו אינו נכלל בטווח.

טיפ פרקטי לכתובת קוד מודרני בתעשייה (2026):

כיום, במקום לכתוב את שם המשתנה פעמיים עם אופרטור **&&**, נהוג להשתמש ב-**Pattern Matching** (תבניות יחס). בצורה המודרנית והנקיה יותר, נכתוב את אותו התנאי בדיוק כך:

```
if (age is >= 40 and < 50)
```

מבחינת הקומפילר הבדיקה זהה לחלוטין, אך התחביר הזה נחשב לקריא ומתוחזק יותר בצוותי פיתוח כיום.

~~~~~

אופרטור לא שווה (**!=**, **not equal**)

אם אנחנו בני 50:

```
int age = 50;
// ==, >, >=, <, <=, != (! =) -> לא שווה
if (age != 43)
```

```
{
    Console.WriteLine("Sorry ,you aren't a great age.");
}
```

הדפסה למסך

```
Console.WriteLine: "Sorry ,you aren't a great age."
```

מכיוון שאנחנו בני 50 ותנאי ה-**if** שלנו בודק שהגיל לא שווה ל-**43**.

אם אנחנו בני 43:

```
int age = 43;
// ==, >, >=, <, <=, != (! =) -> לא שווה
if (age != 43)
{
    Console.WriteLine("Sorry ,you aren't a great age.");
}
```

אין הדפסה למסך **console writeline**.

מכיוון שאנחנו בני **43**.

אנחנו נקבל הדפסה למסך **console writeline** אך ורק כאשר הגיל לא שווה ל-**43**.

~~~~~

## אופרטור || (OR)

במשפטי **||** (או) אם אחד מהצדדים הוא **true**, אז כל המשפט **true**.

כלומר, מספיק שאחד מהם יהיה נכון כדי שכל המשפט ייחשב **true**:

```
if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
```

C#

## הבדל מול אופרטור && (AND)

לא כמו משפטי **&&** (**AND**), שבהם שני הצדדים חייבים להיות **true** כדי שהמשפט יהיה **true**.

```

Console.Write("What is your first name; ");
string firstName = Console.ReadLine();

Console.Write("What is your last name; ");
string lastName = Console.ReadLine();

if (firstName.ToLower() == "tim" &&
    lastName.ToLower() == "corey")
{
    Console.WriteLine("Hello Professor Corey");
}
else if (firstName.ToLower() == "tim" ||
    lastName.ToLower() == "corey")
{
    Console.WriteLine("You have a great part in your name.");
}
else
{
    Console.WriteLine("Hello Student.");
}

int age = 43;

if (age >= 40 && age < 50)
{
    Console.WriteLine("You are in your 40's");
}

```

**נניח שהקלט הוא "Tim" "Corey"**

אנחנו מקבלים:

הדפסה של

"Hello Professor Corey" וגם "You are in your 40's"

**נניח שהקלט הוא "Tim" "Smith"**

אנחנו מקבלים הדפסה :

"You have a great part in your name." וגם "You are in your 40's"

**נניח שהקלט הוא "Sue" "Corey"**

אנחנו מקבלים:

"You have a great part in your name." וגם "You are in your 40's"

**נניח שהקלט הוא "Sue" "Storm"**



"Hello Student." וגם "You are in your 40's"

~~~~~

הבעיה ביצירת תנאים מורכבים ללא הפרדה

אנחנו יכולים להפוך את הבדיקות הללו למורכבות יותר, אבל זה לא רעיון טוב. לדוגמה, אנחנו יכולים לומר:
 גיל (גדול או שווה) $\leq$ ל-40 וגם $\&\&$ גיל (קטן) $>$ מ-50 וגם $\&\&$ גיל (גדול או שווה) $\leq$ ל-70 וגם $\&\&$ גיל (קטן) $>$ מ-80:

```
int age = 43;

if (age >= 40 && age < 50 && age >= 70 && age < 80)
{
    Console.WriteLine("You are in your 40's");
}
```

מדוע הביטוי לעולם לא יחזיר True

הכתיבה הזו לא מומלצת, מכיוון שהביטוי הזה לעולם לא יחזיר **true**.
 הסיבה היא שאם הגיל עכשיו הוא **43** אז הבדיקה הראשונה של גיל $\leq$ ל-40 (גדול או שווה) מחזירה **true**.
 הבדיקה השנייה של גיל $>$ מ-50 (קטן) מחזירה **true**.
 אבל הבדיקה השלישית של גיל $\leq$ ל-70 (גדול או שווה) מחזירה **false**.
 לכן, כל הביטוי הופך ל-**false**.
 גם הבדיקה הרביעית של גיל $>$ מ-80 (קטן) מחזירה **true**, אבל הבדיקה הכללית כבר נכשלה.
 בעקבות זאת, אנחנו עשויים לחשוב שאנחנו צריכים לשים כאן אופרטור **|| (OR)**:

```
int age = 43;

if (age >= 40 && age < 50 || age >= 70 && age < 80)
{
    Console.WriteLine("You are in your 40's");
}
```

הקוד הזה עובד מכיוון שהוא בודק האם הגיל (גדול או שווה) $\leq$ ל-40 וגם (קטן) $>$ מ-50, או האם הגיל (גדול או שווה) $\leq$ ל-70 וגם (קטן) $>$ מ-80. במקרה של גיל 43, אנחנו נקבל בפלט את ההודעה שאנו בשנות ה-40 שלנו.

אם נציב את הגיל 73:

```
int age = 73;

if (age >= 40 && age < 50 || age >= 70 && age < 80)
{
    Console.WriteLine("You are in your 40's or 70's");
}
```

זה עובד ואנחנו מקבלים בפלט את ההודעה המתאימה שאנו בשנות ה-40 או ה-70. אבל בצורה הזו, אנחנו משאירים לקומפיילר להבין לבד מה סדר הפעולות של אופרטורי ה-AND ($\&\&$) וה-OR ($\|\|$) ואיך הם עובדים יחד, וייתכן שלא התכווננו שה-OR הספציפי הזה יתבצע בסדר שהקומפיילר החליט עליו. לכן, אנחנו משתמשים בסוגריים וקובעים במפורש מהו הסדר שבו אנחנו רוצים שהבדיקה תתבצע:

```
int age = 73;

if ((age >= 40 && age < 50) || (age >= 70 && age < 80))
{
    Console.WriteLine("You are in your 40's or 70's");
}
```

עכשיו הקומפיילר יבדוק קודם את הביטוי הראשון (גיל בין 40 ל-50), ואז יבצע את אופרטור ה-OR מול הביטוי השני (גיל בין 70 ל-80).

אבל אם נרצה, נוכל גם לכתוב את זה בצורה אחרת כדי לשנות את הלוגיקה:

```
int age = 73;

if (age >= 40 && (age < 50 || age >= 70) && age < 80)
{
    Console.WriteLine("You are in your 40's or 70's");
}
```

עכשיו אנחנו אומרים:

הגיל $\leq$ ל-40, וגם $\&\&$ (אחד מאלו: גיל $>$ מ-50 או $\|\|$ גיל $\leq$ ל-70) בגלל האופרטור $\|\|$, וגם הגיל $>$ מ-80. אנחנו משתמשים בסוגריים כדי לשנות את קבוצת הבדיקה ואת סדר הערכת התנאים.

הכלל המנחה לכתיבת קוד נכון וקריא

הדרך הטובה ביותר - כאשר יש לנו מספר תנאים, אפילו רק 3 תנאים (אלא אם כן כולם מאותו סוג, כלומר כולם שרשרת של אופרטור && או כולם שרשרת של אופרטור ||) - היא תמיד לאגד אותם באמצעות סוגריים:

```
int age = 73;

if ((age >= 40 && age < 50) || (age >= 70 && age < 80))
{
    Console.WriteLine("You are in your 40's or 70's");
}
```

בצורה הזו אנחנו מאוד ברורים.

לא רק מול הקומפיילר, אלא גם מול כל מתכנת שיבוא אחרינו ויקרא את הקוד - הוא לא יצטרך לנחש למה התכווננו. בנוסף, כדי להפוך את הקוד לקל מאוד לקריאה בעין, מומלץ לכתוב אותו בחלוקה לשורות בצורה הבאה:

```
int age = 73;

if ((age >= 40 && age < 50) ||
    (age >= 70 && age < 80))
{
    Console.WriteLine("You are in your 40's or 70's");
}
```